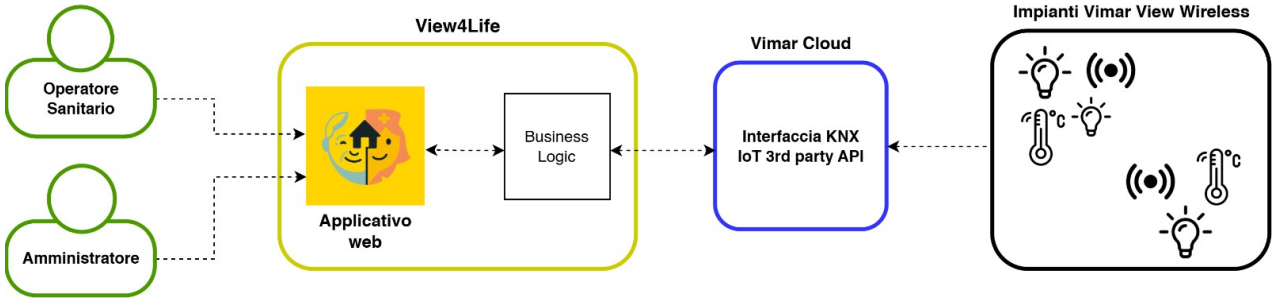
Presentazione architettura

Gruppo 1 - A.A 2025/2026

20/04/2026

Obiettivo capitolato

- Capitolato C9 (Vimar View4Life): Piattaforma per la gestione degli impianti Smart nelle residenze protette;
- Sistema di gestione allarmi e utenti (OSS e Amministrazione) che possono interagire con gli allarmi (gestione e risoluzione)
- Visualizzazione di statistiche sui consumi, con relativi consigli per ridurli, e sugli allarmi.



Architettura di deployment

- Frontend: *Single Page Application* in Angular, servita da *nginx*;
- Backend: monolite realizzato con NestJS.

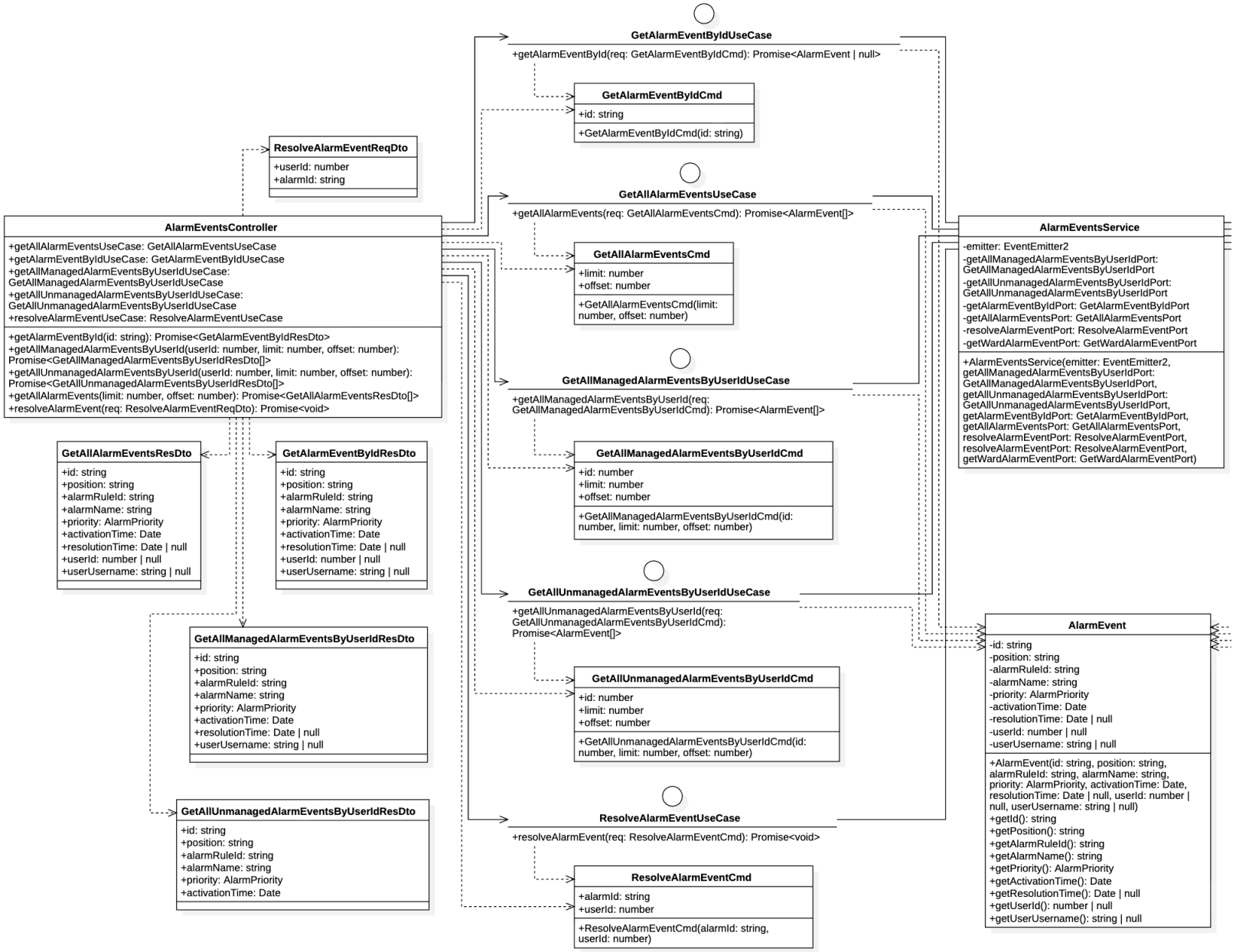
Entrambi, come il database *PostegreSQL*, sono containerizzati e orchestrati da *Docker Compose*.

Attualmente tutti e tre i container sono in esecuzione in un singolo nodo.

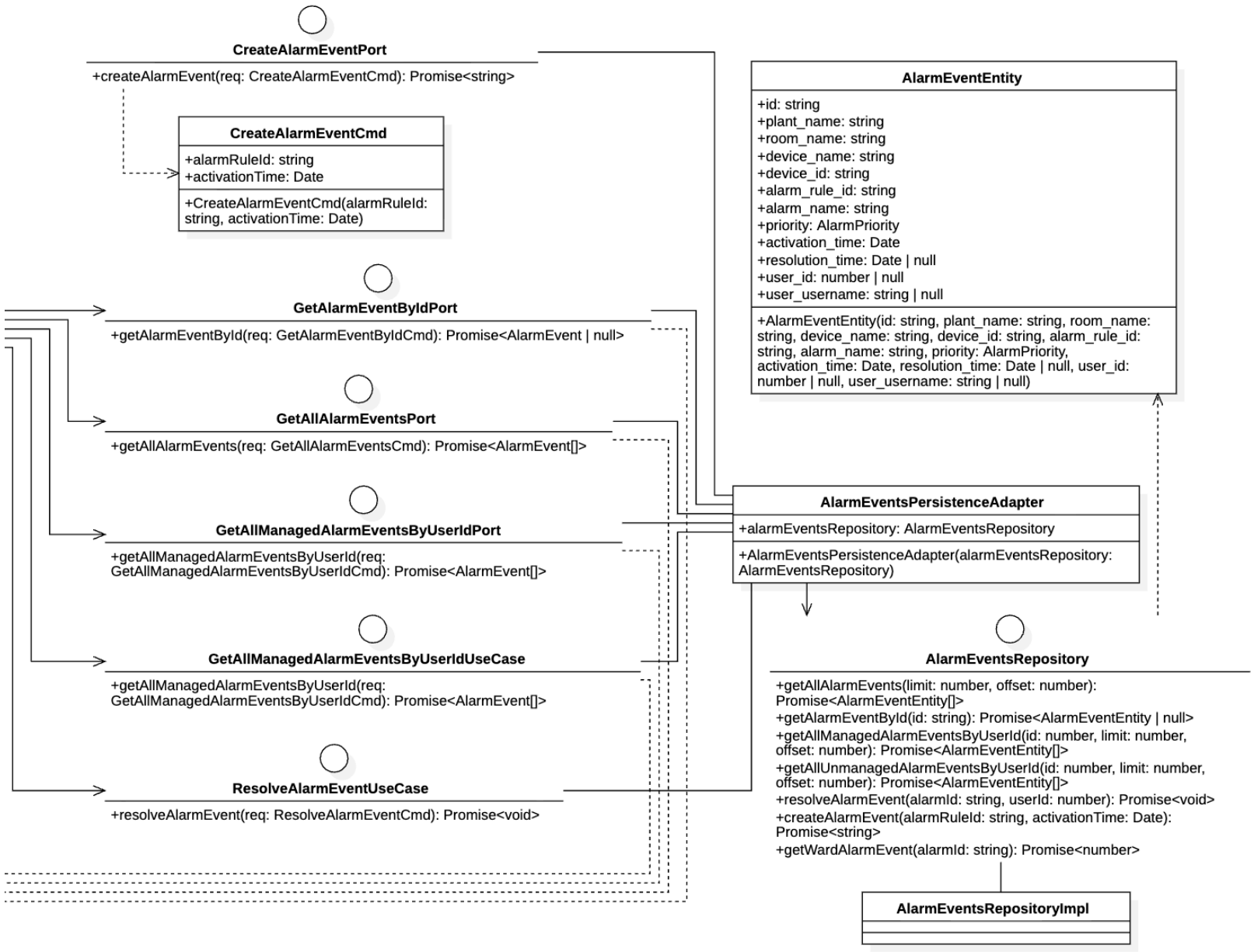
Backend: Architettura esagonale

- Problema:
  - La logica di business del *backend* dipende dai dettagli infrastrutturali (*Persistent Logic*);
  - Difficile testare il *backend* senza dover istanziare l'intera infrastruttura.
- Soluzione: **Architettura Esagonale**
  - Il dominio non dipende da componenti esterne;
  - Le **Ports** sono interfacce *TypeScript* che il dominio espone, gli **Adapters** le implementano;
  - Le dipendenze tra i moduli sono risolte tramite il *pattern* **Dependency Injection**.

UML Alarm (Backend) (1/2)



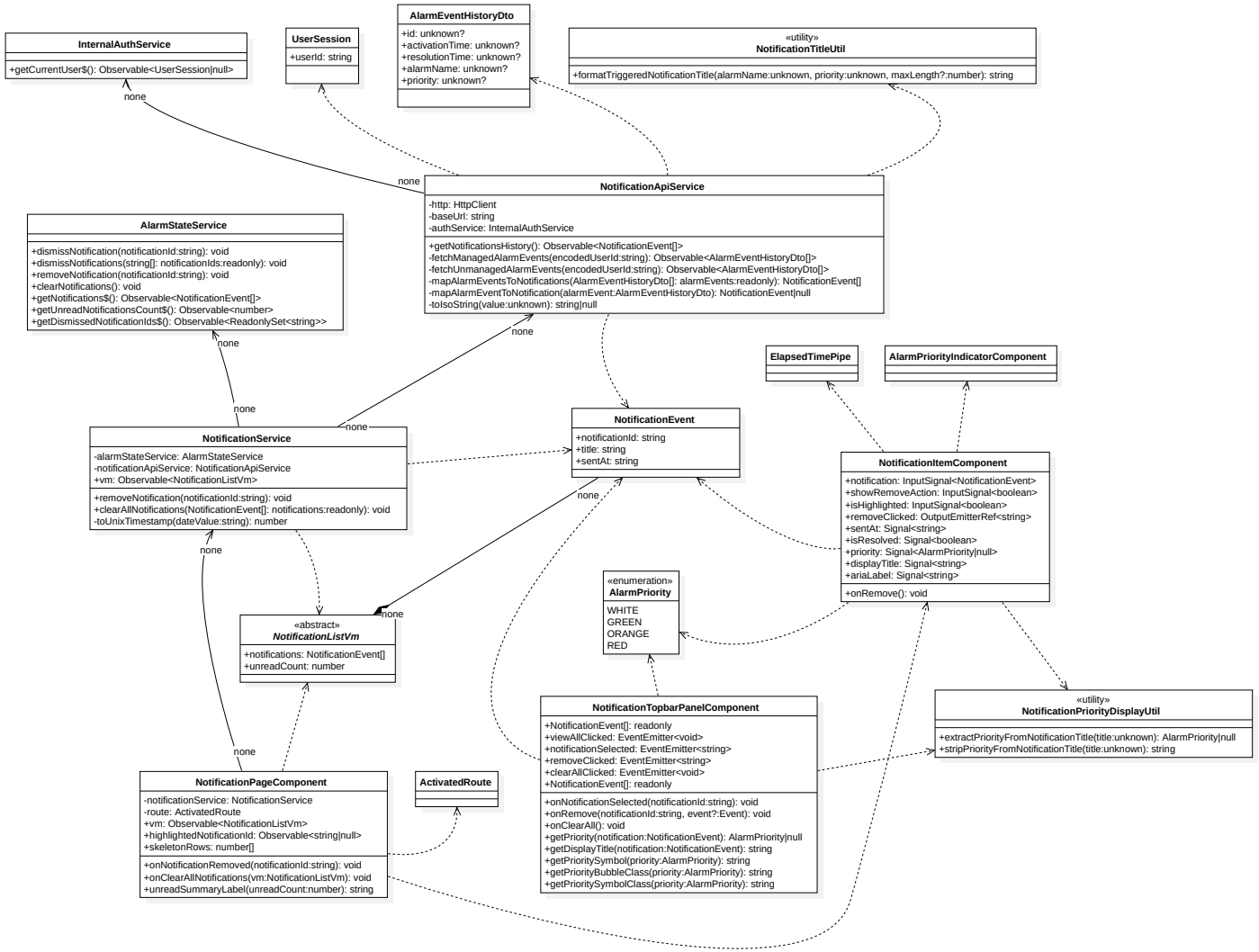
UML Alarm (Backend) (2/2)



Frontend: Architettura Model-View-ViewModel

- Problema:
  - Logica di presentazione legata all'interfaccia utente;
  - Ogni cambiamento della vista rischia di impattare la logica di business.
- Soluzione: **Architettura MVVM** che separa le tre responsabilità:
  - **Model**: dati e logica di dominio, implementati come interfacce/tipi e come servizi (`@Injectable`).
  - **View**: i template HTML del componente; passiva, si limita a realizzare le viste mostrando i dati via **property binding** e a notificare eventi via **event binding**.
  - **ViewModel**: la classe del componente; fa da mediatore tra Model e View, esponendo i dati tramite **Observable** senza contenere logica di dominio.

UML Notification (Frontend)



Code snippets Notification (Frontend)

```
1  @Component({
2    selector: 'app-notification-page-component',
3    changeDetection: ChangeDetectionStrategy.OnPush,
4    imports: [AsyncPipe, NotificationItemComponent],
5    providers: [NotificationService],
6    templateUrl: './notification-page-component.html',
7    styleUrl: './notification-page-component.css',
8  })
9  export class NotificationPageComponent {
10   private readonly notificationService = inject(NotificationService);
11   private readonly route = inject(ActivatedRoute);
12
13   public readonly vm$: Observable<NotificationListVm> = this.notificationService.vm$;
14
15   public onClearAllNotifications(vm: NotificationListVm): void {
16     this.notificationService.clearAllNotifications(vm.notifications);
17   }
```

Figura: notification-page-component.ts

```
1  @if (vm$ | async; as vm) {
2  <div class="notification-list space-y-4">
3      <div class="flex w-full flex-wrap items-center justify-between gap-3">
4          <span
5              class="notification-page__summary inline-flex items-center gap-2">
6              <span class="h-2 w-2 rounded-full bg-amber-500"></span>
7              {{ unreadSummaryLabel(vm.unreadCount) }}
8          </span>
9
10         @if (vm.notifications.length > 0) {
11         <button
12             type="button"
13             class="ml-auto cursor-pointer inline-flex items-center gap-2"
14             (click)="onClearAllNotifications(vm)">
15             Cancella tutte
16         </button>
17         }
18     </div>
```

Figura: notification-page-component

Pattern - Dependency Injection (1/2)

- Problema:
  - I moduli sono fortemente accoppiati tra di loro;
  - Il codice risulta difficile da testare e da manutenere.
- Soluzione: pattern **Dependency Injection**
  - Le dipendenze vengono fornite tramite un *injector*;
  - In *TypeScript (Nest JS)* il pattern è supportato nativamente tramite il decoratore *@Injectable()* e il sistema *providers*.

Pattern - Dependency Injection (2/2)

```
1  providers: [
2      { provide: CREATE_ALARM_RULE_USE_CASE, useClass: AlarmRuleService },
3      { provide: CREATE_ALARM_RULE_PORT, useClass: AlarmRulesPersistenceAdapter },
4      { provide: DELETE_ALARM_RULE_USE_CASE, useClass: AlarmRuleService },
5      { provide: DELETE_ALARM_RULE_PORT, useClass: AlarmRulesPersistenceAdapter },
6      { provide: GET_ALL_ALARM_RULES_USE_CASE, useClass: AlarmRuleService },
```

```
1  @Injectable()
2  export class AlarmRuleService
3    implements
4      CreateAlarmRuleUseCase,
5      GetAlarmRuleByIdUseCase,
6      GetAllAlarmRulesUseCase,
7      UpdateAlarmRuleUseCase,
8      DeleteAlarmRuleUseCase,
9      CheckAlarmRuleUseCase
10 {
```

Pattern - Observer (1/3)

- Problema:
  - I componenti *Angular* devono reagire a eventi asincroni (*WebSocket*) e sincronizzare lo stato *UI*.
- Soluzione: pattern **Observer**
  - Implementato tramite *RxJS*: le componenti si iscrivono a *Observable* senza conoscere la sorgente dei dati;
  - Gli *Observable* gestiscono in modo uniforme eventi asincroni e aggiornamenti di stato *UI*;
  - La comunicazione tra componenti avviene tramite *Subject* e *BehaviorSubject* eliminando ogni accoppiamento diretto.

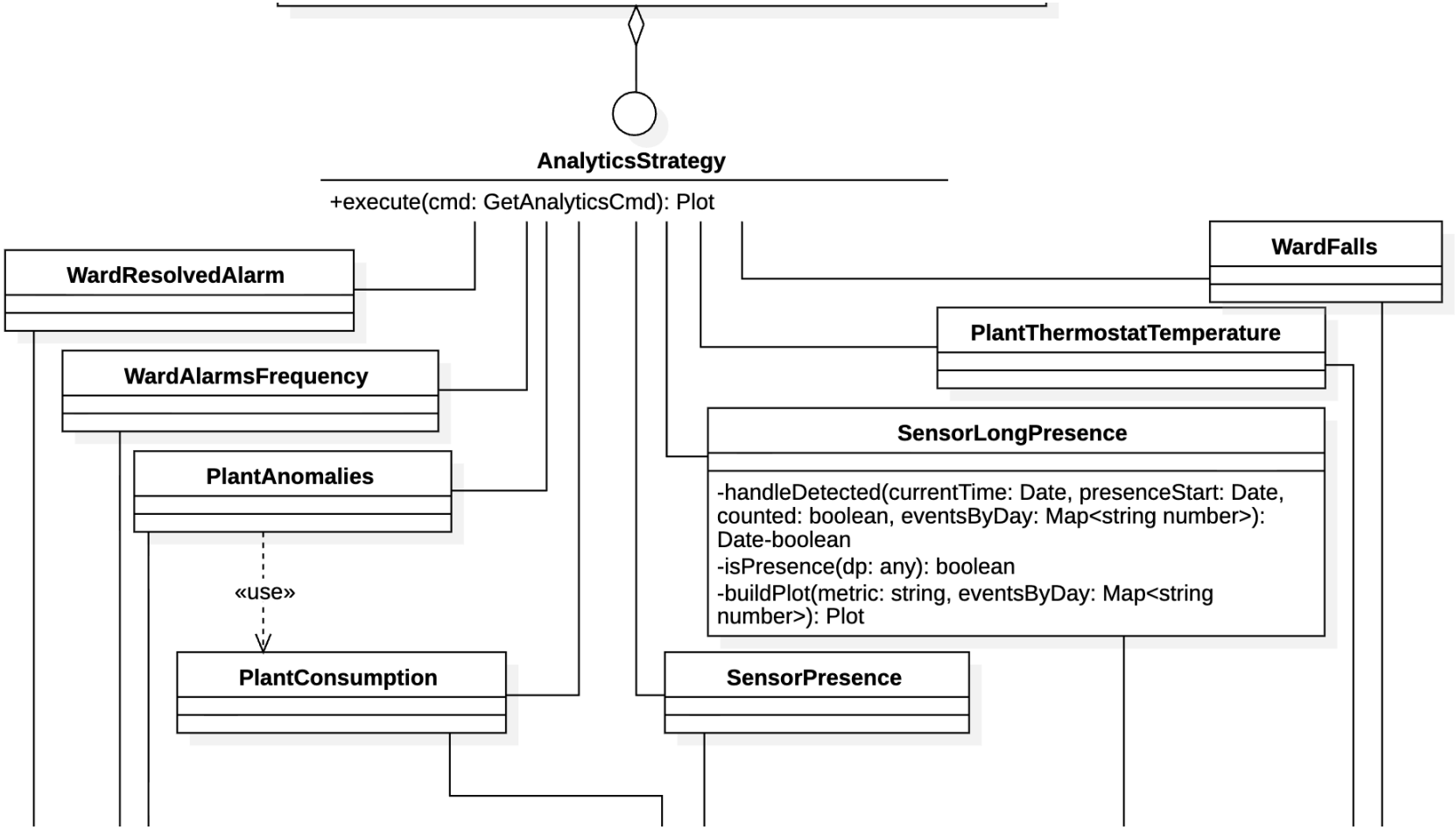
Pattern - Observer (2/3)

UML

Pattern - Observer (2/3)

CODICE

Pattern - Strategy (1/2)

- Problema:
  - Visualizzare un'insieme di *analytics* relative agli impianti;
  - Ogni *analytics* ha un grafico specifico, basato su dati differenti (Es. consumo della luce, temperatura media del termostato, ecc.);
  - Deve essere possibile modificare le *analytics* se necessario.
- Soluzione: **Pattern Strategy**
  - È stata realizzata un'interfaccia `AnalyticsStrategy` che fornisce il metodo `execute`, che le *strategies* implementano.

Pattern - Strategy (2/2)